Routed Is Not Right: A Gold-Master Benchmark for Machine Routing of High-Frequency Printed Circuit Boards

Dave Rauchwerk
Rauchwerk Research
dbrauchwerk@gmail.com

November 2026

Abstract

Learning-based placement and routing for printed circuit boards (PCBs) is usually evaluated on wirelength, via count and design-rule violations. None of the public PCB benchmarks we found scores the properties that make a high-frequency board work: impedance-controlled widths, intra-pair skew and length matching. We introduce PCB Gold-Master Eval, a benchmark of eight fabricated, Apache-licensed boards by Antmicro (4–12 copper layers, 20–797 components, PCIe, MIPI CSI-2, HDMI, GMSL2, DDR4 and DDR5 interfaces). Each task strips the routed copper from a board pinned by commit and checksum, asks an agent or algorithm to route it without moving parts, and scores the result against the human original. Because the designers' length and skew targets are not written in any rule file, the verifier derives them from the human copper itself. Tasks are packaged in the Harbor format; an oracle that restores the human copper scores 1.00 on 8 of 8 tasks. A Freerouting 2.5 baseline completes 72–96% of connections on boards without dense BGAs and stalls on a BGA board, and scores far below the human boards on skew, length and width conformance. We pair the benchmark with a graded review of thirty sources, a scored inventory of open-source tools, and a parametric forecast of benchmark reward against compute budget and wall-clock time, which suggests that for open-source stacks the evaluator, not compute, limits progress. Code, tasks and forecast are released under Apache-2.0.

1 Introduction

Automated routers have long completed PCBs; the hard part of a modern board is completing it *correctly* at high frequency. A DDR5 byte lane, a PCIe link or a MIPI CSI-2 lane works only if its traces keep a controlled impedance on every layer, the two halves of each differential pair arrive within a few picoseconds of each other, and groups of signals are matched in length. Experienced designers meet these constraints by hand, often with serpentine meanders tuned to fractions of a millimetre.

Recent work proposes that machine learning—reinforcement learning (RL), evolutionary search, graph networks and large language models (LLMs)—can learn to place and route boards for such constraints without hand-written heuristics. The evidence is thin. The best-known RL placement result [14] remains disputed after an independent reassessment [4, 6, 13]. Published PCB routing benchmarks score wirelength, vias and design-rule checks (DRC) [18], and none of the learned PCB methods we found optimises signal integrity (SI) end to end. A method cannot be shown to work on a property that is not measured.

This paper contributes:

1. **A gold-master benchmark.** Eight fabricated open-hardware boards, stripped of copper, with the human layout as the answer key and a reward that includes pair skew, length-group spread and impedance-width conformance (Section 3).

2. **Reproducible packaging.** Harbor tasks [2] with every input pinned (image digest, board commit, file checksums), a separate verifier sandbox, an oracle, and an integrity signal for runs that copy the public gold layout (Section 4).

3. **Baselines and a compute forecast.** A Freerouting baseline on the full set, and a parametric model of reward against dollars and hours built from published scaling evidence and measured costs (Sections 5–6).

2 Related work

Chip placement with learning. Graph-based RL for macro placement [14] reported human-competitive floorplans; an independent study found simulated annealing equal or better at roughly one twentieth of the CPU-hours, with weak rank correlation between the proxy cost and routed wirelength or timing [4]. Black-box and tree-search methods [5, 17] and offline RL [11] outperform earlier RL placers [10] at hour scale, and GPU analytical placement [12] remains fast and strong. ChiPBench [20] showed that leading on intermediate metrics need not improve final power, performance and area—the central motivation for scoring against end results.

PCB placement and routing. RL placement of PCB components with adaptive rewards reduced post-route wirelength by 17–21% against simulated annealing [19]. Ranking Cost learns cost maps for an A* router with evolution strategies [7]. PCBWorld provides a KiCad-based RL routing environment over 679 real boards, where policies trained on synthetic boards approach rule-based routers on easy splits but trail them on harder ones [18]. Koppar's autorouter-bench compares routers on public KiCad boards, including several by Antmicro, by open connections, DRC errors and time [9]. None of these scores impedance, skew or length matching.

LLM-driven program evolution. FunSearch [16] and AlphaEvolve [15] evolve programs scored by automated evaluators. In placement, LLM-evolved optimisers improved half-perimeter wirelength by 5–8% [22], and HELIX evolved placement heuristics inside OpenROAD for 4.5% (global) and 5.6% (routed) geometric-mean wirelength gains at roughly $250–300 per run [8].

SI surrogates. Deep networks predict PDN impedance >100× faster than boundary-element simulation [23], and buffer-parameterised surrogates predict eye metrics across technologies with large speedups over simulation [21]. Their generalisation beyond trained topologies is not yet established. Antmicro's gerber2ems turns fabrication files into openEMS full-wave models [3].

3 Benchmark

3.1 Gold-master boards

We selected eight KiCad boards from Antmicro's open-hardware portfolio (Table 1). Selection favoured (i) real high-speed interfaces, (ii) evidence that the board was fabricated and used—measured VNA data, photographs, commercial sale or published research use—and (iii) a spread of

Board	Tier	Cu	Parts	Pair nets	To route	Interfaces
SI test board	easy	4	28	2	107	50 Ω lines
M.2 PCIe adapter	easy	4	20	18	93	PCIe x4
HDMI–MIPI bridge	easy	4	121	36	304	HDMI, CSI-2
GMSL deserializer	medium	4	236	28	413	GMSL2, CSI-2
Jetson Orin baseboard	medium	8	680	208	1,625	PCIe, USB3, CSI-2
RDIMM DDR4 tester	hard	12	691	226	2,021	DDR4
SO-DIMM DDR5 tester	hard	12	706	102	1,770	DDR5
RDIMM DDR5 tester	hard	12	797	178	2,300	DDR5

Table 1: The gold-master set. *To route*: unconnected connections after stripping, from KiCad's connectivity engine. Tiers reflect layer count, size and interface difficulty.

size. All are Apache-2.0. Boards are fetched at pinned commits and verified by SHA-256; nothing is redistributed.

A notable property of the set is that none of Antmicro's rule files (`.kicad_dru`) contains length or skew limits; they define per-layer widths and gaps for impedance classes only. The matching the designers achieved lives only in their copper—for example, the DDR4 tester carries 5,514 hand-drawn meander arcs. Scoring against the copper therefore captures intent that no rule file states.

3.2 Task

Each task provides a board with all tracks, arcs, vias, tuning patterns and zone fills removed, with placement, footprints, nets, net classes, zone outlines and rules unchanged. The agent must save a routed board in place without moving, rotating, flipping, adding or deleting footprints. The environment supplies KiCad 10.0.7 (with `pcbnew` Python bindings), Java 25, Freerouting 2.5.0 [1] and network access for other tools.

3.3 Reward

Let u_0, u_g and u_c be the unconnected counts of the stripped, gold and candidate boards after a zone refill. The reward is

$$R = P \cdot C^2 \cdot (0.30\,D + 0.25\,S + 0.15\,L + 0.15\,W + 0.15\,E)\,, \tag{1}$$

where $P \in \{0, 1\}$ gates on untouched placement; $C = 1 - (u_c - u_g)/(u_0 - u_g)$ is completion; $D = 1/(1 + x/10)$ penalises x DRC errors in excess of the gold board's own, by type; S is the fraction of differential pairs whose skew is within $\max(0.15\,\text{mm}, 1.5\times$ the gold skew); L is the fraction of non-default net classes whose length spread is within $\max(1.25\times, +1\,\text{mm})$ of the gold spread; W is the fraction of controlled-impedance copper whose width matches a width the designers used for that class on that layer (within $5\,\mu$m); and E averages the gold-to-candidate ratios of total copper length and via count, capped at 1. Squaring C keeps unfinished boards from scoring well on the quality terms. The verifier also reports F, the fraction of candidate segments coincident with gold segments on a $10\,\mu$m grid; independent routing essentially never reproduces human geometry, so runs with $F > 0.5$ are disqualified from leaderboards.

Board	Time (s)	Compl.	DRC	Skew	Length	Width	Eff.	Reward
SI test board	54	0.90	1.00	0.00	1.00	0.00	1.00	**0.48**
M.2 PCIe adapter	112	0.96	0.18	0.11	0.00	0.43	1.00	**0.27**
HDMI–MIPI bridge	181	0.18	0.34	0.00	0.00	0.00	1.00	**0.01**
GMSL deserializer	534	0.72	0.08	0.29	0.00	0.00	1.00	**0.13**

Table 2: Freerouting 2.5 against the gold masters (1.00 = the human original). Columns are the reward terms of Section 3.

4 Implementation

Tasks follow the Harbor format [2]: `instruction.md`, `task.toml`, an environment Dockerfile, an oracle `solve.sh` and a verifier `test.sh`. The Dockerfile fetches the gold board, strips it and deletes it in a single image layer, so the gold copper exists in no layer the agent can read. The verifier runs in a separate sandbox, re-fetches the gold board at the pinned commit, checks its checksum, refills zones and scores. Two engineering findings are worth recording. First, `kicad-cli`'s DRC report stops listing unconnected items at 499, so completion must be counted with KiCad's connectivity engine instead; a stripped DDR5 board has 2,300. Second, removing items through the `pcbnew` SWIG bindings crashed in KiCad 10.0.7, so stripping is done on the normalised text representation. All tools, pins and the forecast are driven by a single `Makefile`.

5 Results

Oracle. Restoring the human copper scores $R = 1.00$ on 8 of 8 tasks through Harbor 0.25.0, confirming that the verifier accepts the answer key on every board, including the 12-layer DDR boards.

Freerouting baseline. Table 2 reports the best of 18 Freerouting runs (pass limits 1, 3 and 10; two seeds) on each board with results so far. Runs used one CPU core under x86 emulation on an Apple-silicon laptop. On the HDMI–MIPI bridge, Freerouting stalled at 181 unrouted items in its own count after one and after three passes, leaving most of the board's BGA fan-out unrouted; published DDR3 BGA routing cases report the same limitation. Results also vary between runs: on the M.2 adapter, reward ranged from 0.21 to 0.27 across seeds and pass limits. Independent runs overlapped the gold copper by about 1% (F between 0.008 and 0.012), far below the disqualification threshold. Where it completes the board, the router still fails the high-frequency terms: on the M.2 PCIe adapter it made 92% of connections, met the skew tolerance on one of nine pairs and used an impedance-correct width on 40% of high-speed copper.

6 A compute forecast

To guide investment we model the reward a strategy reaches with budget B and wall clock T. Each strategy has a fixed cost (B_0, T_0), a per-candidate cost c and duration τ, and a parallelism limit p; the number of candidates is $N = \min\left((B - B_0)/c,\ (T - T_0)\,p/\tau\right)$ and

$$G(N) = g_0 + (g_{\max} - g_0)\left(1 - (1 + N/n_0)^{-\alpha}\right). \tag{2}$$

With $\alpha = 0.6$ and $n_0 = 60$ the curve reproduces WireMask-EA's observation that 73% of the final gain arrives within the first 300 of 2,000 evaluations [17]. Starting points come from the

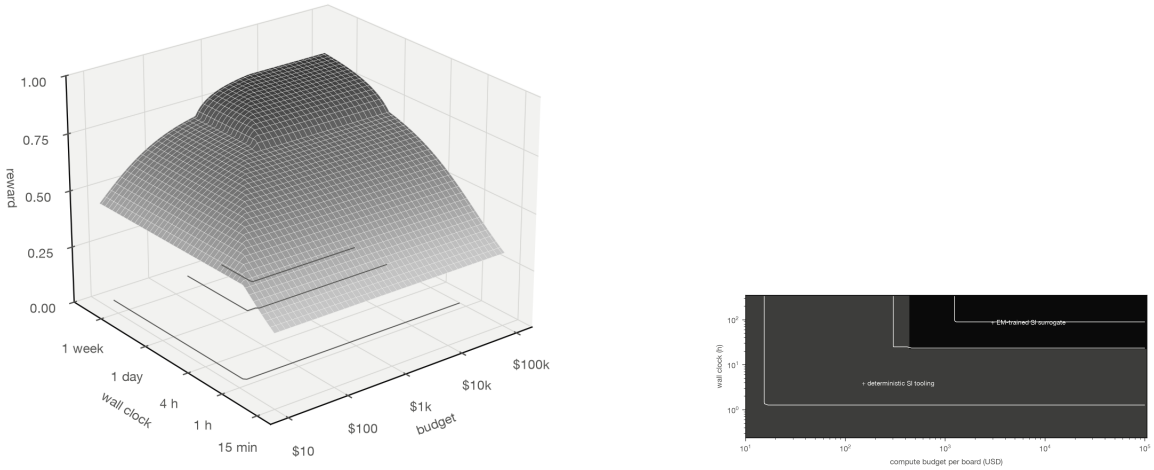


Figure 1: Forecast reward on the hard tier (DDR5-class board). Left: best achievable reward over seven strategies against budget and wall clock. Right: the strategy that achieves it.

Budget	4 h	1 day	1 week
$100	0.71 / 0.61	0.78 / 0.66	0.78 / 0.66
$1,000	0.71 / 0.61	0.82 / 0.70	0.84 / 0.79
$10,000	0.71 / 0.61	0.82 / 0.70	0.84 / 0.82

Table 3: Forecast reward (medium / hard tier) by budget per board and wall clock.

measured baselines; per-candidate costs from the baselines and on-demand prices on 7 November 2026; LLM-evolution costs from HELIX [8]; RL fixed costs from the reassessment of Cheng et al. [4]; and ceilings for untested strategies are explicit assumptions, tagged in the released parameter file. Figure 1 shows the best achievable reward on the hard tier and Table 3 summarises both tiers.

Three implications follow. Routing candidates are cheap—about one cent of CPU for a medium board—so budgets above about $100 per board barely move simple search; the binding constraints are wall clock and evaluator fidelity. Strategies that improve the evaluator (deterministic SI tooling and EM-trained surrogates) are the only ones whose ceiling rises with spend. RL, with a fixed cost of several thousand dollars and days, reaches less than search within the horizons considered. These are forecasts, not results; the benchmark exists to replace them with measurements.

7 Limitations

The set has eight boards from one design house, which limits stylistic diversity. The boards are public and may appear in model training data; the coincidence signal F detects literal copying but not memorised style. The reward uses geometric proxies for SI; it does not yet run full-wave simulation, which gerber2ems makes possible on sampled nets. Length groups are approximated by net class. Baselines ran under emulation on one machine and will be repeated on native hardware. Placement is held fixed; placement tasks are future work.

8 Conclusion

Routing a board and routing it right are different problems, and the second has lacked a public measuring stick. Using fabricated open boards as gold masters turns the claims of self-improving layout systems into numbers anyone can reproduce. We invite submissions of agents, routers and learned methods.

Reproducibility. Code, tasks, baselines, forecast parameters and this paper's figures are at `https://github.com/elevenarms/pcb-gold-eval`; `make all` rebuilds them from pinned inputs.

Acknowledgements. The gold-master boards are Antmicro's open-hardware designs. Research for this paper was assisted by AI agents; all citations were verified against primary records.

References

[1] Freerouting: an advanced autorouter for all PCB programs. `https://github.com/freerouting/freerouting`, 2026. Version 2.5.0.

[2] Harbor: a framework for evaluating and optimizing agents and models in container environments. `https://github.com/harbor-framework/harbor`, 2026. Version 0.25.0.

[3] Antmicro. gerber2ems: PCB signal-integrity simulation with openEMS. `https://github.com/antmicro/gerber2ems`, 2026.

[4] Chung-Kuan Cheng, Andrew B. Kahng, Sayak Kundu, Yucheng Wang, and Zhiang Wang. Assessment of reinforcement learning for macro placement. In *Proceedings of the International Symposium on Physical Design (ISPD)*, 2023. doi: 10.1145/3569052.3578926. Extended version: arXiv:2302.11014.

[5] Zijie Geng et al. Reinforcement learning within tree search for fast macro placement. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 15402–15417, 2024.

[6] Anna Goldie, Azalia Mirhoseini, and Jeff Dean. That chip has sailed: A critique of unfounded skepticism around AI for chip design, 2024.

[7] Shiyu Huang, Bin Wang, Dong Li, Jianye Hao, Ting Chen, and Jun Zhu. Ranking cost: Building an efficient and scalable circuit routing planner with evolution-based optimization, 2021.

[8] Andrew B. Kahng, Sayak Kundu, and Bodhisatta Pramanik. HELIX: Placement heuristics evolution via LLM-instructed code exploration. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD)*, 2026. doi: 10.1145/3831252.3834232.

[9] Koppar AI. autorouter-bench. `https://github.com/Koppar-AI/autorouter-bench`, 2026.

[10] Yao Lai, Yao Mu, and Ping Luo. MaskPlace: Fast chip placement via reinforced visual representation learning, 2022.

[11] Yao Lai et al. ChiPFormer: Transferable chip placement via offline decision transformer. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, 2023. arXiv:2306.14744.

[12] Yibo Lin et al. DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement. In *Proceedings of the Design Automation Conference (DAC)*, 2019. doi: 10.1145/3316781.3317803.

[13] Igor L. Markov. Reevaluating Google's reinforcement learning for IC macro placement. *Communications of the ACM*, 2024. doi: 10.1145/3676845.

[14] Azalia Mirhoseini, Anna Goldie, et al. A graph placement methodology for fast chip design. *Nature*, 594:207–212, 2021. doi: 10.1038/s41586-021-03544-w.

[15] Alexander Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025.

[16] Bernardino Romera-Paredes et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.

[17] Yunqi Shi, Ke Xue, Lei Song, and Chao Qian. Macro placement by wire-mask-guided black-box optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.16844.

[18] Hyungseok Song, Junseok Park, Won-Seok Choi, Seohui Bae, Han-Seul Jeong, Youngjoon Park, and Soonyoung Lee. PCBWorld: A benchmark environment for engine-grounded PCB design automation, 2026. KDD 2026 workshop (non-archival).

[19] Luke Vassallo and Josef Bajada. Learning circuit placement techniques through reinforcement learning with adaptive rewards. In *Design, Automation and Test in Europe (DATE)*, 2024. doi: 10.23919/DATE58400.2024.10546526.

[20] Zhihai Wang et al. Benchmarking end-to-end performance of AI-based chip placement algorithms, 2024.

[21] Julian Withoft, Werner John, Emre Ecik, Ralf Bruning, and Jurgen Gotze. Buffer-parameterized machine learning surrogate models for cross-technology signal integrity analysis and optimization, 2026.

[22] Xufeng Yao, Jiaxi Jiang, Yuxuan Zhao, Peiyu Liao, Yibo Lin, and Bei Yu. Evolution of optimization algorithms for global placement via large language models, 2025.

[23] Ling Zhang, Jack Juang, Zurab Kiguradze, Bo Pu, Shuai Jin, Songping Wu, Zhiping Yang, and Chulsoon Hwang. Fast PDN impedance prediction using deep learning, 2021.